

FORMAN CHRISTIAN COLLEGE (A CHARTERED UNIVERSITY)



COMP 451 B

SP24

Lab 5

Hafsah Shahbaz – 251684784

Daim Bin Khalid - 251686775

INTRODUCTION

This lab report delves into the implementation of a C program designed to detect comments in C source files. The program aims to reinforce core concepts in compiler pre-processor, C programming, particularly focusing on file handling, character manipulation, and command-line argument parsing.

Objectives:

- Implement file handling operations to read and analyze text files.
- Perform character manipulation to identify comments both inline and block.
- Utilize command-line argument parsing to specify source code files to detect comments.

The program analyzes two C source files specified by the user as command-line arguments. It identifies two types of comments: single-line comments starting with `"/"` and multi-line comments enclosed between `"/*" and "*/"`. Upon execution, the program displays the contents of each file and removes the comments where they are found.

Standard C library functions serve as the backbone of this program, facilitating various tasks such as file handling, input validation, and character manipulation. These functions streamline development, ensuring efficient and reliable program execution.

1. **<stdlib.h>** This library provides functions for memory allocation, process control, and other utility functions. In our program, it's utilized for process control.
 - `exit()`: This function terminates the program immediately, closing all open files and releasing allocated memory. It's invoked in error handling routines to gracefully exit the program in case of invalid inputs or file operations.
2. **<stdio.h>** The standard input-output library in C provides functions for file handling and console input and output operations. In our program, it's extensively used for file handling and console output.
 - `fopen()`: Opens a file specified by its filename and mode. It's utilized to open the input file in read mode for reading its content, as well as for error checking to ensure the file exists and can be accessed.
 - `fgetc()`: Reads a character from the specified file. It's used to read characters from the input file one by one.
 - `fprintf()`: Writes formatted output to a file. It's employed to write the line numbers and modified content to the output file.
 - `printf()`: Used to format and print output to the console. It's employed for displaying informative messages, usage instructions, and file content during program execution.
 - `fclose()`: Closes the file associated with the given file pointer. This function is invoked to release system resources after reading the file content or performing decoding operations.

Each function serves a distinct purpose within the program, contributing to its overall functionality and effectiveness.

ALGORITHM and LOGIC

The program begins by verifying the presence of the correct number of command-line arguments. If the expected number of arguments is not provided, the program displays the correct usage format and exits. Following this, the program executes two functions, `detectSlashComments()` and `detectStarComments()`, passing the provided source file paths as arguments.

- The program takes three command-line arguments: executable name, source code file to detect slash comments in, and source code file to detect star comments in.
- The `detectSlashComments()` function examines each line of the source file to identify double-slash comments. It opens the file in read mode and reads each line sequentially. Within each line, it searches for occurrences of `"/"` indicating the beginning of a double-slash comment. If found, it prints the line number and a message indicating the presence of such a comment. The function then proceeds to the next line until the end of the file is reached.
- `detectStarComments()` function analyzes each line of the source file to locate block comments enclosed between `"/" and "/"`. It opens the file in read mode and iterates through each line, searching for occurrences of these opening and closing symbols. When an opening symbol is found, it stores the line number in a stack to later match with the corresponding closing symbol. If a closing symbol is found, it retrieves the corresponding opening line number from the stack and prints a message indicating the presence of the block comment. The function continues this process until the end of the file is reached.

Further breakdown of code:

1. `detectSlashComments()` Function:

Functionality:

This function is responsible for detecting double slash comments in a given source file. It reads the content of the file line by line, searching for instances where two consecutive slashes `"/"` appear. If such a pattern is found, it prints the line number along with a message indicating the presence of a double slash comment.

Arguments:

- `sourceFile`: A pointer to a constant character string representing the path of the source file to be analysed.

Variables:

- `readFile`: FILE pointer to handle the source file for reading.
- `buffer`: Character array to store each line of the file read from the source file.
- `line`: Integer variable representing the line number being processed.

Code Snippet:

```
void detectSlashComments(const char *sourceFile)
{
    printf("Source file: %s\n", sourceFile);
    displayFile(sourceFile);

    FILE *readFile = fopen(sourceFile, "r");
    if (readFile == NULL)
    {
        perror("Failed to open file");
        return;
    }

    char buffer[256];
    int line = 1;

    printf("\n");
    while (fgets(buffer, 256, readFile) != NULL)
    {
        for (int i = 0; buffer[i] != '\0'; i++)
        {
            if (buffer[i] == '/' && buffer[i + 1] == '/')
            {
                printf("Line Number: %d          A double slash comment found\n", line);
            }
        }

        line++;
    }

    fclose(readFile);
}
```

Explanation:

- This function starts by printing the name of the source file being analyzed and then displays its content using the displayFile() function.
- It opens the source file in read mode and checks if the file opened successfully. If not, it prints an error message and returns.
- Using fgets(), it reads each line of the file into the buffer and then iterates through each character of the line.

- Within the inner loop, it checks for the presence of a double slash comment pattern '//'. If found, it prints the line number and a message indicating the detection of a double slash comment.
- After processing all lines of the file, it closes the file.

2. detectStarComments () Function:

Functionality:

This function is responsible for detecting block comments in a given source file. It reads the content of the file line by line, searching for instances where '/*' and '*/' appear, indicating the beginning and end of block comments, respectively. It maintains a stack to keep track of opening comment symbols and ensures that closing symbols are matched appropriately. After processing the entire file, it checks for any unclosed block comments and prints a warning message if found.

Arguments:

- `sourceFile`: A pointer to a constant character string representing the path of the source file to be analysed.

Variables:

- `readFile`: FILE pointer to handle the source file for reading.
- `buffer`: Character array to store each line of the file read from the source file.
- `openingStack`: Array to store line numbers where opening comment symbols '/*' are encountered.
- `stackPointer`: Integer variable to keep track of the top of the `openingStack` array.
- `line`: Integer variable representing the line number being processed.

Code Snippet:

```
void detectStarComments(const char *sourceFile)
{
    printf("Source file: %s\n", sourceFile);
    displayFile(sourceFile);

    FILE *readFile = fopen(sourceFile, "r");
    if (readFile == NULL)
    {
        perror("Failed to open file");
        return;
    }

    char buffer[256];
    int openingStack[256];
    int stackPointer = 0;
    int line = 1;

    printf("\n");
    while (fgets(buffer, 256, readFile) != NULL)
    {
        for (int i = 0; buffer[i] != '\0'; i++)
        {
            if (buffer[i] == '/' && buffer[i + 1] == '*')
            {
                printf("Opening symbol found in line %d\n", line);
                openingStack[stackPointer] = line;
                stackPointer++;
                i++;
            }
            else if (buffer[i] == '*' && buffer[i + 1] == '/')
            {
                if (stackPointer != 0)
                {
                    int openingNumber = openingStack[stackPointer - 1];
                    printf("Closing symbol against opening symbol in line %d found in line %d\n", openingNumber, line);
                    stackPointer--;
                }
            }
        }
    }
}
```

```

        i++;
    }
}

line++;
}

if (openingStack[0] != 0)
{
    printf("Closing symbol against opening symbol for line");
    for (int i = 0; i < stackPointer; i++)
    {
        printf(" %d", openingStack[i]);
        if (i + 1 != stackPointer)
        {
            printf(", and");
        }
    }
    printf(" not found\n");
}

fclose(readFile);
}

```

Explanation:

- This function starts by printing the name of the source file being analyzed and then displays its content using the `displayFile()` function.
- It opens the source file in read mode and checks if the file opened successfully. If not, it prints an error message and returns.
- Using `fgets()`, it reads each line of the file into the buffer and then iterates through each character of the line.
- Within the inner loop, it checks for the presence of opening comment symbols `'/'` and closing comment symbols `'/'`.
- It maintains a stack to keep track of opening comment symbols encountered and ensures that closing symbols are matched appropriately.
- After processing all lines of the file, it checks for any unclosed block comments and prints a warning message if found.

3. displayFile () Function:

Functionality:

This function displays the content of a file on the console.

Arguments:

- `sourceFile`: Name of the file to be displayed.

Variables:

- `readFile`: File pointer for the file.

Code Snippet:

```
void displayFile(const char *sourceFile)
{
    FILE *readFile = fopen(sourceFile, "r");
    if (readFile == NULL)
    {
        perror("Failed to open file");
        return;
    }
    int c;

    while ((c = fgetc(readFile)) != EOF)
    {
        printf("%c", c);
    }

    printf("\n");
    fclose(readFile);
}
```

Explanation:

- The function opens the file specified by filename in read mode.
- It reads characters from the file one by one and prints them to the console until the end of file (EOF) is reached.
- Finally, it closes the file.

main () Function:

Functionality:

This function serves as the entry point of the program. It is responsible for orchestrating the execution of other functions by processing command-line arguments, opening files, invoking functions to detect different types of comments in source files, and displaying results on the console.

Arguments:

- `argc`: An integer representing the number of command-line arguments passed to the program.
- `argv`: An array of strings containing the command-line arguments.

Variables:

- `argc`: Integer variable storing the number of command-line arguments.
- `argv`: Array of strings storing the command-line arguments. Arguments passed will be source files to remove comments from.

Code Snippet:

```
int main(int argc, char **argv)
{
    if (argc != 3)
    {
        printf("Usage: %s <sourceFile1.c> <sourceFile2.c>\n", argv[0]);
        return 1;
    }

    detectSlashComments(argv[1]);
    printf("\n");
    detectStarComments(argv[2]);
}
```

Explanation:

- This function begins by checking if the correct number of command-line arguments (three) is provided. If not, it prints the correct usage and exits with error code 1.
- If the correct number of arguments is provided, it proceeds to invoke the `detectSlashComments` function with the first source file as an argument to detect double slash comments.
- After detecting double slash comments, it prints a newline character for formatting purposes.
- Then, it invokes the `detectStarComments` function with the second source file as an argument to detect block comments.
- Finally, it returns 0 to indicate successful execution.

OUTPUT

1. Sample Run 1:

Input:

- Input file (sample.c & sample2.c):

sample.c:

```
//prg1.c
```

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    printf("Hello class\n");//this is a printing statement
```

```
    //end of program
```

```
    return 0;
```

```
}
```

sample2.c:

```
/*prg1.c*/
```

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    /* this is an input file
```

```
       for task 2s*/
```

```
    printf("Hello class\n");
```

```
    return 0;
```

```
}
```

Result:

```
● daim@da-pc:~/projects/compiler/lab5$ ./main sample.c sample2.c
Source file: sample.c
//prg1.c
#include <stdio.h>
int main()
{
    printf("Hello class\n");//this is a printing statement
    //end of program
    return 0;
}

Line Number: 1          A double slash comment found
Line Number: 5          A double slash comment found
Line Number: 6          A double slash comment found

Source file: sample2.c
/*prg1.c*/
#include <stdio.h>
int main()
{
    /* this is an input file
       for task 2s*/
    printf("Hello class\n");
    return 0;
}

Opening symbol found in line 1
Closing symbol against opening symbol in line 1 found in line 1
Opening symbol found in line 5
Closing symbol against opening symbol in line 5 found in line 6
Closing symbol against opening symbol for line not found
```

Comments:

- The 2 input files contain slash comments and star comments in them respectively.
- The code files are displayed.
- The program detects where the comments are and prompts them to the user.
- Star comments are checked where they are completed or left without closing.

2. Sample Run 2:

Input:

- Input file (sample3.c):

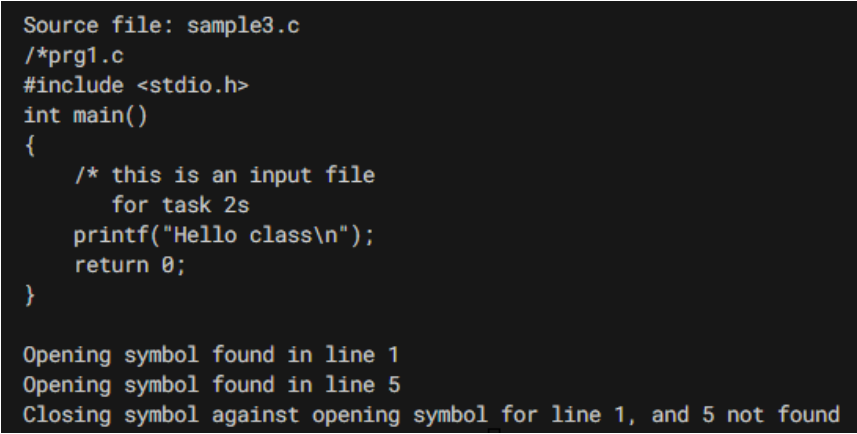
sample3.c:

```
/*prg1.c
```

```
#include <stdio.h>
```

```
int main()
{
    /* this is an input file
       for task 2s
    printf("Hello class\n");
    return 0;
}
```

Result:



```
Source file: sample3.c
/*prg1.c
#include <stdio.h>
int main()
{
    /* this is an input file
       for task 2s
    printf("Hello class\n");
    return 0;
}

Opening symbol found in line 1
Opening symbol found in line 5
Closing symbol against opening symbol for line 1, and 5 not found
```

Comments:

- Demonstration of star comment left unclosed being detected.

3.Sample Run 3:

Input:

- Input file (sample4.c):

sample4.c:

```
/*prg1.c*/
#include <stdio.h>
int main()
```

```
{  
    /* this is an input file  
       for task 2s  
    printf("Hello class\n");/*  
    this is a comment*/  
    return 0;  
}
```

Result:

```
Source file: sample4.c  
/*prg1.c*/  
#include <stdio.h>  
int main()  
{  
    /* this is an input file  
       for task 2s  
    printf("Hello class\n");/*  
    this is a comment*/  
    return 0;  
}  
  
Opening symbol found in line 1  
Closing symbol against opening symbol in line 1 found in line 1  
Opening symbol found in line 5  
Opening symbol found in line 7  
Closing symbol against opening symbol in line 7 found in line 8  
Closing symbol against opening symbol for line 5 not found
```